# An implementation of Open-RMF in an industrial environment

Michele Belletti

2025
December

# Contents

# Part I

# Introduction

This work is a study on a new system OPEN-RMF that is still in active development but has reached a good state to apply for the robots in the facility Elettra Sincotrone Trieste

# Chapter 1

# Background and motivation

Elettra Sincontrone Trieste[1] is a big research facility that, principally, gives to users two light sources: the synchrotron Elettra and the laser at free electron Fermi. Besides this, it study new technology and technique to improve the their capabilities and work activities. A lot of the latest improvements are based on robots, that also after the covid and latest with the exploding diffusion of LLMs, allow an interaction of the users of the light sources to communicates and interact with the robots in the facility to obtain information and services as transport of packages. They already developed a global planner D-Star (D-star based optimized trajectory planner) [5] in ROS 1 for sending a robot in the infrastructure that is optimized for exploring a big space. But ROS 1 has reached the end of life so we need to rewrite the code using the ROS 2 ecosystem. Furthermore, because there are multiple vendor that are specialized for different type of work, there is the problem of the interaction between them. For this in the latest years is born Open-RMF[2] a multi platform for control and manage a multi fleet float of robots.

[1] https://www.elettra.eu/
[2] https://www.open-rmf.org/

# Chapter 2

# Research questions and objectives

Our objects for this thesis is: given the company Mini RoverRobotics[1] Robot

**Figure 2.1:** Rover Mini of the RoverRobotics

import it to ROS 2 Nav2 ecosystem using a porting of the global planner D-star from ROS 1 to ROS 2[2] and then add it to Open-RMF to control it. Compare it to the ROS 1 stack and validate the use of the Open-RMF ecosystem in Elettra Sincotrone Trieste.

[1]https://roverrobotics.com/

# Chapter 3

# Scope and limitations

The principal limitation is the now non accessible control for the lifts that will not enable us to have a direct control on them and also there is not an automatic open-close control over the doors. Even if the hight customability of open-rmf is possible to use a human interaction for active these systems, like when the robot reach a door, it the send a message or a light can be turned on and then wait until someone will open the door or a lift. But is not a good approach.

# Part II

# Literature Review

In this part we will provide a description of the technologies used in this thesis. We will start describing what is ROS and how it works then we will talk about the navigation system used and how it is implemented the control with the fleet management system Open-RMF

# Chapter 4

# Overview of ROS1 and ROS2

The Robotics Operating System [1] is a compilation of tools and libraries open-source that help the development of application in the robotic environment. otherwise from the name, it's not a Operating System (OS) but an Software Development Kit (SDK) that gives at the user all the tools for creating your robot giving some typically functionality of an OS: hardware abstraction, low-level device control, implementation of commonly-used functionality, message-passing between processes, and package management. The ROS ecosystem is based on (all these things is going to be described in detail in the next section):

- An communication system named middleware or plumbing based on DDS (Data Distribution Service). This allow the Nodes to share information between each other with an anonymous publish/subscribe pattern allowing fault isolation, separation of concerns and clear interfaces and a synchronous system (remote procedure call)RPC-style communication between services.
- Tools to control and improve the development of the software like launch, introspection, debugging, visualization, plotting, logging, and playback.
- Provides tools and libraries for obtaining, building, writing, and running code across multiple computers.
- A active and big community with at the center the Open Robotics[2].

## 4.1 Structure of ROS

The documentation[3] give all the information of ROS, some functionality write below are not available for ROS 1:

- The **ROS Graph** is a peer-to-peer network of processes that elaborated simultaneously and communicate with each other.
- **Nodes** is a modular process and should be a specific resolution for a problem to be more easily communicate with other Nodes through topics, services, action and parameters. They also can be based on LifecycleNode that contain a state

[1] https://www.ros.org/
[2] https://www.openrobotics.org/
[3] https://docs.ros.org/en/rolling/index.html

machine transitions for bringup and teardown to allow more deterministic behavior.

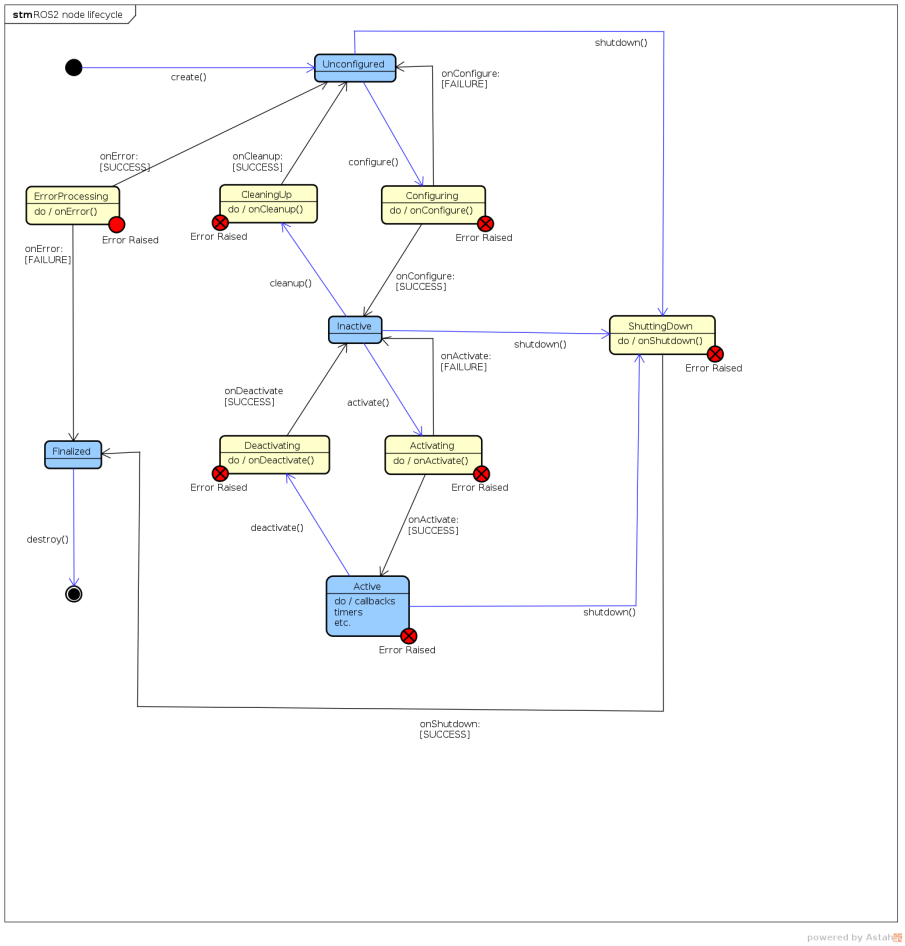


**Figure 4.1:** State Machine of LifecycleNode

- **Topics** are the bus for communication between Nodes to share a streams of messages in a publisher to subscriber structure.

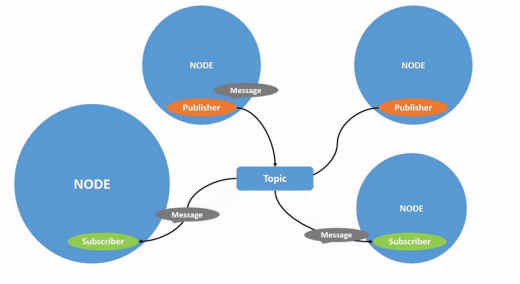


**Figure 4.2:** Topic process

- **Service** are based on the request and response model. When a node with a Service client send a Request Message to Node with a Service Server it will send a Response Message to that specific Service Client.

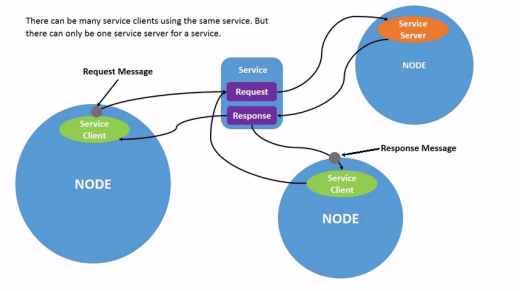


**Figure 4.3:** Service process

- **Parameters** are configuration of a node and allow to modified them without changing the code.
- **Actions** were created to fulfill the needs of something for tracking long task. They are a combination of two service (goal and result) and a pub-sub (feedback). It send a goal request and receive a response then send a result request, the node then start to pub to the feedback topic and then response to the result request.

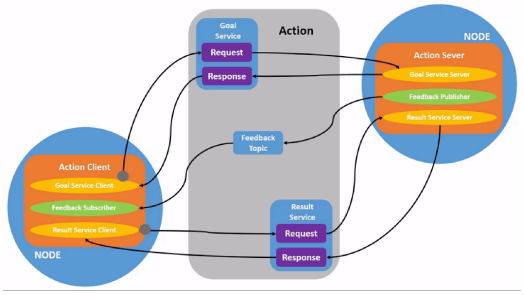


**Figure 4.4:** Action process

## 4.2 Tools of ROS

Now i will describe the tools that are built in ROS for developing robotics system:

- **Launch** is a file structure that allow to start multiple nodes with a set of parameters.
- **rqt** is a graphical user interface (GUI) tool for ROS 2. With a series of plugins allow to interact with an interface for helping in the development, in the monitoring and diagnostic the ROS system.
- **rqt_console** visualize the log generate by the ROS 2 system.
- **Rviz** is a 3D visualizer for the Robot Operating System framework?.
- **rosdep** is a dependency installer utility.
- **sros2** is a package that provide the tools and instruction to apply security on DDS.

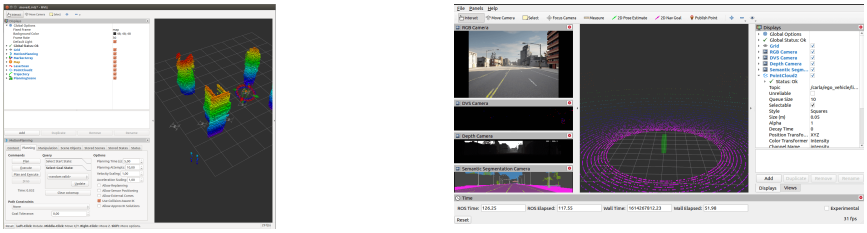

**Figure 4.6:** Examples of visualization of ROS data with Rviz

Another tools that are connected with ROS and helps develop is the simulation environments. They allow to test the code on a simulated robot in different condition to find possible bugs in the code that if there was launched on a real robot can be harmful to it or the environment. They provide realistic physical mechanics at all the parts of simulation and in the documentation of ROS there are listened 2: Gazebo and Webots. The first one is also a project under the Open Robotics and for a lot of projects is the default used like the one we will use. Gazebo offer:

- **ROS Integration** a bridge to converts the protobuf messages to ROS.
- **Performance tools** that allow to have it distributed on servers, dynamically asset loading and control of the time step for simulation to have the best performances.
- **Realistic simulation** that allow to insert a variety of sensor with their noise in a 3D rendering word built on Ogre 2.1[4] and with the physic engine DART[5].
- **Extensibility** because its built on modularity, it allow with plugins to run code that interact with the simulation.

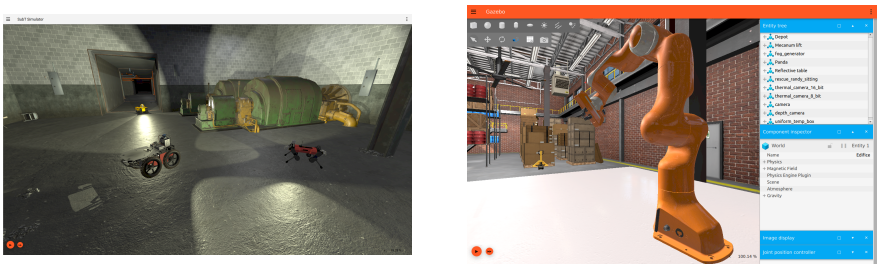

**Figure 4.8:** Examples of Gazebo

[4]https://wiki.ogre3d.org/Home
[5]https://dartsim.github.io/

# Chapter 5

# Introduction to NAV2 and Google Cartographer

Multiple tools are available for move a robot in an environment. The one that give a full system is Nav2 [1] [3] the successor of ROS Navigation Stack of ROS 1.

## 5.1 Description of Nav2

Nav2 gives all the tools (perception, planning[4], control, localization, visualization) for whichever robot to navigate autonomously and reliably also thought hard environment. It allow to build a model of the environment from the data of the sensors, calculated dynamically the best path to follow to avoiding obstacles using a map, set the velocity of the motors and allow to follow logic step using a behaviors tree.

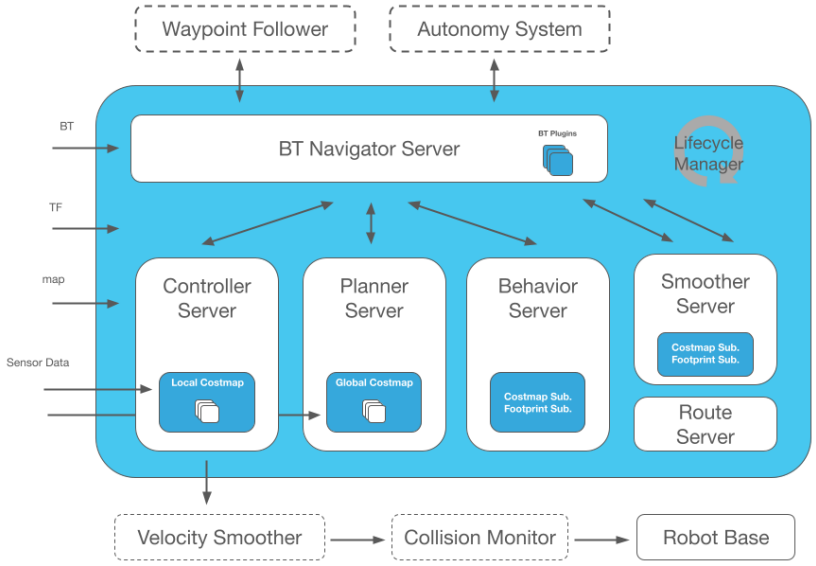


**Figure 5.1:** Nav2 architecture

These are also some of the updates that have changed from ROS 1 other less notable changes are the use of:

[1] https://docs.nav2.org/

- **Behavior trees**[1]: gives to Nav2 a structure for completing a series of tasks in a human readable way. Their structure is similar to a tree and allow to create complex system that can be tested with different tools. The package used for this is BehaviorTree.CPP[2] that respect others allow to link different tree.

- **nav2_util LifecycleNode** is a wrapper of rclcpp_lifecycle LifecycleNode of the standard structure of ros as described in Nodes 4.1 and add a bond connection to the lifecycle manager, node that knows the states of each node, and allow to transition down its lifecycle nodes for safety if some node has crashed.

- **Action Server**: the functionality are described in the structure of ROS 4.1 and they are implemented in Nav2 for the interaction with the Behavior Tree (BT) like NavigationToPose (move to a location), NavigationThroughPoses (allow to select multiple waypoints), etc.

The control is based on this nodes:

- **controller_server** is the node that handle the request for moving the robot, using tree plugins progress checker (check if the robot has make some progress in following the path), goal checker (check if the robot has reached the goal position) and controller (generate the velocity command using the **local costmap** to move the robot avoiding collision)

- **planner_server** is the node that handle the request for calculate the path, using a loaded plugin planner, to reach a position using the **global costmap** and the data from sensors.

- **bt_navigator** is a Behavior Tree-based implementation of navigation that implement the NavigateToPose, NavigateThroughPoses, and other task interfaces to increment the flexibility during navigation.

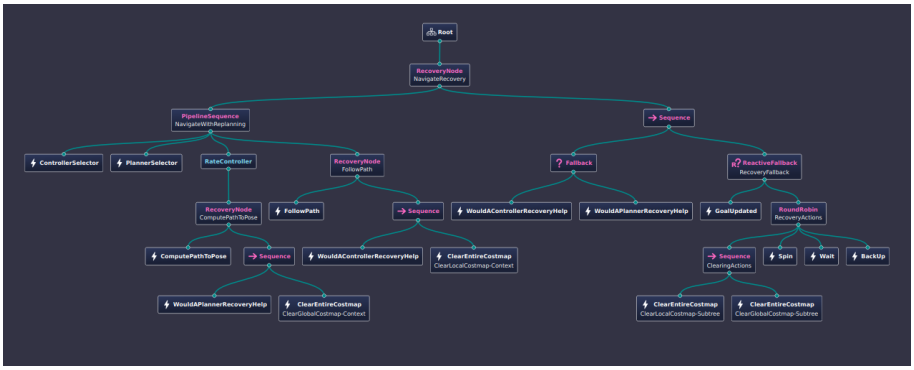


**Figure 5.2:** Nav2 bt navigation TOCECK

- **smoother_server**

- **behavior_server**

- **waypoint_follower**

[2]https://www.behaviortree.dev/

- **velocity_smoother**
- **global_costmap**
- **local_costmap**

## 5.2 Cartographer

# Chapter 6

# Review of related work on porting navigation stack from ROS1 to ROS2

# Chapter 7

# Overview of open source fleet management systems, particularly Open-RMF

## 7.1 Open-RMF description

...

today the tool for converting the data are https://www.ros.org/blog/noetic-eol/
implement also planning benchmark

# Bibliography


[1] Michele Colledanchise and Petter Ögren. *Behavior Trees in Robotics and AI*. July 2018. DOI: 10.1201/9780429489105. URL: http://dx.doi.org/10.1201/9780429489105.

[2] Steven Macenski et al. "Robot Operating System 2: Design, architecture, and uses in the wild". In: *Science Robotics* 7.66 (2022), eabm6074. DOI: 10.1126/scirobotics.abm6074. URL: https://www.science.org/doi/abs/10.1126/scirobotics.abm6074.

[3] Steven Macenski et al. "The Marathon 2: A Navigation System". In: *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2020.

[4] DV Lu A. Merzlyakov M. Ferguson S. Macenski T. Moore. "From the desks of ROS maintainers: A survey of modern & capable mobile robotics algorithms in the robot operating system 2". In: *Robotics and Autonomous Systems* (2023).

[5] Andrey Vukolov et al. "Universal Configurable Navigation and Control System for Industrial Unmanned Ground Vehicles with Differential Chassis". In: *Advances in Mechanism and Machine Science*. Ed. by Masafumi Okada. Cham: Springer Nature Switzerland, 2023, pp. 287–301. ISBN: 978-3-031-45770-8.